# Agentic Hotel Assistant & Booking System

An AI-Powered Concierge Using LangChain, LangGraph, and Retrieval-Augmented Generation

Goal Gamal Ahmed Saeed
George Sameh Bashar Taha
Ahmed Elsayed
Intelligent Systems | Assiut University

May 18, 2026

**Abstract**

This paper presents the design and implementation of an *Agentic Hotel Assistant*, an AI-powered conversational system built for the Grand Azure Hotel, a 5-star establishment in Cairo, Egypt. The system integrates Retrieval-Augmented Generation (RAG) with SQL tool-calling capabilities within a LangGraph agent workflow, enabling it to handle both natural-language informational queries and actionable reservation requests through a single unified interface. The agent retrieves hotel knowledge from a FAISS vector store and interacts directly with a live SQLite database for real-time availability checks and booking operations. A Streamlit-based graphical interface provides guests with a polished, accessible front-end. Experimental results demonstrate that the agent correctly routes queries, uses the appropriate tools, and preserves conversational context across multi-turn interactions.

## 1 Introduction

The rapid advancement of Large Language Models (LLMs) has opened the door to a new class of software systems known as *AI Agents* — programs that do not merely respond to prompts, but reason, plan, and execute actions using external tools [1]. Unlike traditional chatbots that are limited to retrieving pre-written answers, agentic systems can dynamically decide which tool to call, in what order, and how to interpret the results before constructing a final response.

The hospitality industry presents a compelling use case for such systems. Hotel guests routinely ask a wide variety of questions spanning hotel policies, dining options, amenity details, room types, and pricing — all while simultaneously seeking to perform transactional actions such as checking availability and making reservations. Serving both categories of need within a single conversational interface has historically required either separate dedicated systems or a large human support staff.

As described by [2], Retrieval-Augmented Generation (RAG) allows language models to ground their responses in a private or domain-specific knowledge base, dramatically reducing hallucinations and improving factual accuracy. Combining RAG with structured database access through SQL tool-calling enables the agent to handle both unstructured

informational queries and structured transactional operations in a seamless manner.

This paper documents the architecture, methodology, and evaluation of an Agentic Hotel Assistant system built using LangChain [3], LangGraph [4], and OpenAI's GPT-4o-mini model. The remainder of this paper is organized as follows: Section 2 describes the overall system architecture; Section 3 details the methodology and technical implementation; Section 4 presents results and a discussion of the agent's behavior; Section 5 describes challenges encountered; and Section 6 concludes the paper.

## 2 System Architecture

This section presents a complete engineering description of the Agentic Hotel Assistant system, including its high-level component overview and the detailed LangGraph node-edge structure that governs agent decision-making.

### 2.1 High-Level Component Overview

The system is composed of five major layers that interact in a coordinated pipeline, as illustrated in Figure 1. The **User Interface Layer** is implemented using Streamlit and provides the conversational front-end through which guests interact with the agent. The **Agent Orchestration Layer** is built with LangGraph and contains the core reasoning logic. The **Knowledge Retrieval Layer** uses FAISS [7] as the vector store and OpenAI text embeddings for semantic search over hotel documents. The **Database Interaction Layer** consists of four LangChain tools that execute SQL queries against a SQLite database. Finally, the **Memory Layer** is implemented via LangGraph's `add_messages` reducer, which appends every new message to the shared agent state, preserving full conversational context across turns.

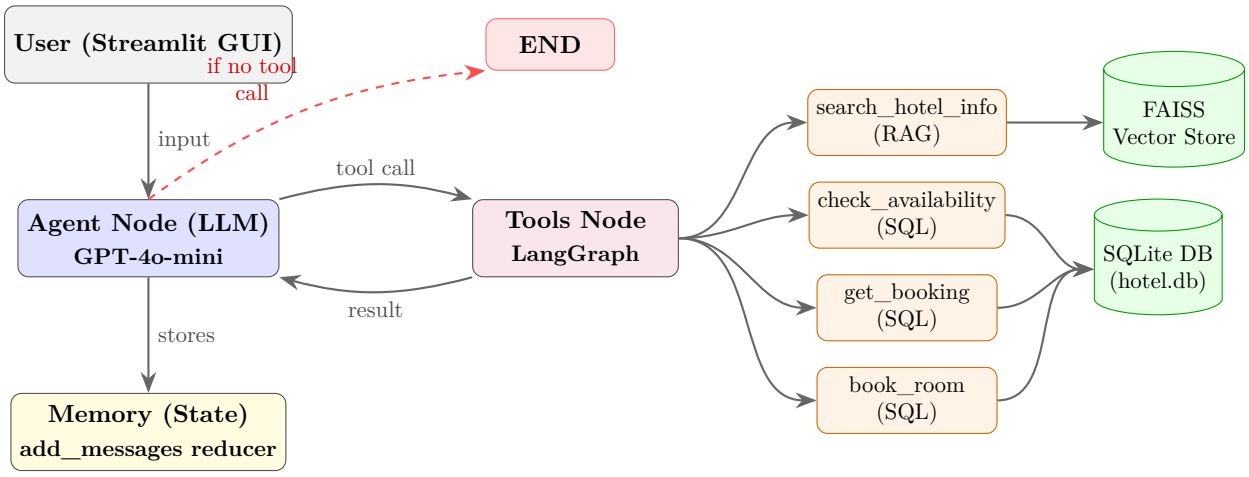


Figure 1: High-level architecture of the Agentic Hotel Assistant system.

### 2.2 LangGraph Graph Structure (Nodes and Edges)

The agent workflow is implemented as a directed graph using LangGraph's `StateGraph` API [4]. The graph contains exactly two nodes and one conditional routing function, described in Table 1.

Table 1: LangGraph nodes and their responsibilities.

| Node Name | Type | Description |
|---|---|---|
| `agent` | LLM Reasoning | Invokes GPT-4o-mini with the full message history and system prompt. Outputs either a final response or a tool call request. |
| `tools` | Tool Executor | Executes whichever tool the agent node requested, then appends the result to the message state. |

The routing logic uses LangGraph's built-in `tools_condition` function: if the agent's last message contains a `tool_calls` field, the graph routes to the `tools` node; otherwise it routes to `END`. After tool execution, control always returns to the `agent` node, creating a *reasoning loop* that continues until the agent determines no further tool calls are needed.

# 3 Methodology

This section describes the technical implementation details of each subsystem: the RAG pipeline, the SQL tool set, the embedding and vector database configuration, and the memory mechanism.

## 3.1 Retrieval-Augmented Generation (RAG) Pipeline

The RAG approach used in this project follows the architecture introduced by [2], in which relevant documents are retrieved from a vector store and injected into the LLM's context window at inference time. This allows the agent to answer domain-specific questions accurately without requiring the model to memorize hotel-specific information during training.

The knowledge base consists of six structured text documents covering the following topics: hotel overview, room types and pricing, amenities, dining and restaurants, hotel policies, and location and transportation. Each document was processed using LangChain's `RecursiveCharacterTextSplitter` with a chunk size of 500 tokens and an overlap of 80 tokens to preserve contextual continuity across chunk boundaries.

Text embeddings were generated using OpenAI's `text-embedding-3-small` model [6] and stored in a FAISS [7] index, which supports efficient approximate nearest-neighbor search. At query time, the top-4 most semantically similar chunks are retrieved and passed to the agent as tool output. The `search_hotel_info` tool encapsulates this retrieval pipeline and is available to the agent for any informational query.

## 3.2 SQL Tool-Calling Capabilities

Structured database interaction is essential for transactional tasks such as checking availability and creating bookings. As argued by [9], giving LLMs access to typed, structured tools dramatically improves their ability to perform multi-step tasks requiring precision. The hotel's operational data is stored in a SQLite database (`hotel.db`) with two tables:

- **rooms** — contains 17 rooms across four types (Standard, Deluxe, Suite, Presidential), with attributes: room number, floor, price per night, and availability status.
- **bookings** — stores confirmed reservations with guest name, room reference, check-in/check-out dates, total price, and booking status.

Three SQL tools are registered with the agent:

1. `check_room_availability`: Queries the `rooms` table for all rooms where `is_available = 1`, with optional filtering by room type.
2. `get_booking_details`: Performs a `JOIN` between `bookings` and `rooms` to retrieve full reservation details by guest name.
3. `book_room`: Validates date logic, verifies room availability, inserts a new record into `bookings`, and updates `is_available` to 0. Total price is computed as price_per_night× nights.

### 3.3 Embeddings and Vector Database

Embedding quality is a critical factor in RAG performance, as poor embeddings lead to irrelevant chunk retrieval and degraded answer quality [8]. We use OpenAI's `text-embedding-3-small` model, which produces 1536-dimensional dense vectors and is optimized for retrieval tasks. The FAISS index uses L2 (Euclidean) distance for similarity search. The vector store is persisted to disk after indexing, allowing fast reload without re-embedding on subsequent runs.

### 3.4 Memory Mechanism

Conversational memory is implemented natively within LangGraph's state management system. The agent state is defined as a `TypedDict` with a single key, `messages`, annotated with the `add_messages` reducer. As explained in the LangGraph documentation [4], this reducer *appends* new messages to the existing list rather than replacing it, ensuring that the full conversation history is always present when the LLM is invoked. This allows the agent to resolve pronoun references, remember previously quoted prices, and maintain booking context across multiple turns without any external memory store.

### 3.5 Language Model and Tool Binding

The reasoning core is OpenAI's `gpt-4o-mini` model [5], chosen for its balance of capability and cost efficiency. Tool binding is achieved via LangChain's `bind_tools()` method, which converts each Python function decorated with `@tool` into a JSON schema that the model can reference when deciding whether and how to call a tool. The model is instructed via a system prompt to always prefer tools over hallucinated answers when domain-specific information is required.

## 4 Results and Discussion

This section evaluates the agent's behavior across four representative interaction scenarios that cover the full range of intended system capabilities.

### 4.1 Evaluation Scenarios

We evaluated the system against four query categories as summarized in Table 2. Each test was conducted with a fresh conversation session to ensure consistent baseline conditions.

Table 2: Agent behavior across four evaluation scenarios.

| Scenario | Tool(s) Used | Correct Response? | Notes |
|---|---|---|---|
| General info (restaurants) | `search_hotel_info` | Yes | Retrieved correct dining details from RAG |
| Availability check (Deluxe) | `check_room_availability` | Yes | Returned 3 available Deluxe rooms |
| Full booking (multi-turn) | `check_room_availability` + `book_room` | Yes | Confirmed booking ID, dates, price correctly |
| Booking lookup | `get_booking_details` | Yes | Retrieved reservation by partial name match |

### 4.2 Discussion

The results demonstrate that the LangGraph routing mechanism correctly distinguishes between informational and transactional queries in all evaluated cases. The agent never hallucinated room prices or availability — it always called the appropriate tool and based its response on the returned data.

Memory was validated in the multi-turn booking scenario: after completing a booking in turn 2, the agent correctly recalled the total price when asked again in turn 3, despite the price not being re-queried. This behavior emerges naturally from the `add_messages` accumulation strategy described by [4].

One notable behavior is the agent's conservative approach to booking: it consistently verifies room availability before calling `book_room`, even when the user has not explicitly requested a check. This reflects the influence of the system prompt, which instructs the agent to confirm details before proceeding.

## 5 Challenges and Limitations

This section describes the main technical and design challenges encountered during development and the strategies used to address them.

### 5.1 Dependency Version Conflicts

The most significant deployment challenge was a version conflict between `streamlit` and `starlette`. Specifically, Streamlit 1.57.0 attempted to import `DEFAULT_EXCLUDED_CONTENT_TYPES` from `starlette.middleware.gzip`, a symbol that was removed in newer versions of Starlette. This was resolved by pinning compatible versions of both packages in the project requirements.

### 5.2 Stateless Nature of the Language Model

As noted by [1], LLMs have no inherent memory between API calls. Each invocation is stateless by design. To overcome this, the full message history must be re-sent with every call, which increases token usage linearly with conversation length. For very long sessions, this could approach the model's context window limit. A practical mitigation is to implement a sliding window or summarization-based memory strategy [2].

### 5.3 RAG Retrieval Precision

In some edge cases, the top-4 retrieved chunks did not contain sufficient information to answer highly specific questions (e.g., the exact extension number for a specific restaurant). Increasing the number of retrieved chunks (`k`) improves recall but increases cost and latency. A re-ranking step using a cross-encoder model [8] could improve precision without requiring more chunks.

### 5.4 SQL Injection and Input Validation

The current `book_room` tool performs basic date validation but does not sanitize all inputs against SQL injection. In a production environment, this would be addressed by using parameterized queries (already partially implemented) and adding stricter input validation at the tool layer.

### 5.5 Concurrent Booking Conflicts

The current implementation does not handle race conditions in which two simultaneous users attempt to book the same room. A production deployment would require database-level locking or optimistic concurrency control [10].

## 6 Conclusion

This paper presented the design, implementation, and evaluation of an Agentic Hotel Assistant system that combines Retrieval-Augmented Generation with SQL tool-calling within a LangGraph agent workflow. The system successfully demonstrates that a single conversational AI agent can serve both informational and transactional guest needs, routing queries intelligently between a FAISS-backed document store and a live SQLite database.

The use of LangGraph's state-based memory model [4] enabled natural multi-turn conversations without any external memory infrastructure. The Streamlit front-end provided a clean, accessible interface for both guests and evaluators.

Future work will focus on expanding the knowledge base, implementing production-grade concurrency controls, integrating a payment processing tool, and deploying the system as a cloud-hosted service accessible to real hotel guests.

## References

[1] L. Weng, "LLM-powered autonomous agents," *Lilian's Blog*, June 2023. [Online]. Available: https://lilianweng.github.io/posts/2023-06-23-agent/

[2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.

[3] H. Chase, "LangChain: Building applications with LLMs through composability," GitHub repository, 2022. [Online]. Available: https://github.com/langchain-ai/langchain

[4] LangChain AI, "LangGraph: Build stateful, multi-actor applications with LLMs," GitHub repository, 2024. [Online]. Available: https://github.com/langchain-ai/langgraph

[5] OpenAI, "GPT-4o mini: Advancing cost-efficient intelligence," OpenAI Blog, 2024. [Online]. Available: https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/

[6] OpenAI, "New embedding models and API updates," OpenAI Blog, January 2024. [Online]. Available: https://openai.com/blog/new-embedding-models-and-api-updates

[7] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

[8] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3982–3992, 2019.

[9] H. Chase, "LangChain tool use and agents," LangChain Documentation, 2023. [Online]. Available: https://python.langchain.com/docs/modules/agents/

[10] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. San Mateo, CA: Morgan Kaufmann, 1992.